

Disciplina:
Arquiteturas de Softwares

Prof. Esp. Leonardo Dias
Curso de Sistemas de Informação

Contêineres

Dia 10

Disciplina: Arquiteturas de Softwares

Arquitetura não é só “um desenho bonito”, mas sim a estrutura que sustenta o software.

Docker: Imagens e Contêineres

Projeto Exemplo: contêineres presentes neste projeto

Neste projeto, que contém uma aplicação **front-end** e uma **back-end**, há **dois contêineres principais**. Cada um é responsável por executar uma parte da aplicação de forma isolada e independente.

Estrutura geral do projeto

```
/projeto
├── /api      → aplicação back-end (Node.js / Express)
│   ├── Dockerfile
│   ├── server.js
│   └── package.json
├── /front    → aplicação front-end (React, Vue ou HTML/JS)
│   ├── Dockerfile
│   ├── index.html (ou /src)
│   └── package.json
└── docker-compose.yml
```

Contêiner 1 - Back-End (API)

O primeiro contêiner executa o **back-end**, desenvolvido em **Node.js**.

Esse contêiner é construído a partir de uma imagem base que contém o ambiente necessário para rodar aplicações JavaScript no servidor.

Sua função é:

- Receber e processar requisições HTTP enviadas pelo front-end.
- Aplicar regras de negócio.

- Acessar e manipular dados, geralmente armazenados em um banco de dados.
- Retornar respostas no formato JSON.

O back-end é um serviço que roda internamente dentro do contêiner, normalmente em uma porta específica (como 3000), e responde às requisições enviadas pelo outro contêiner.

Contêiner 2 - Front-End

O segundo contêiner executa o **front-end**, que é a camada responsável pela interface com o usuário.

Esse contêiner é baseado em uma imagem que contém um servidor web leve, usado para disponibilizar os arquivos estáticos da aplicação (como HTML, CSS e JavaScript).

Suas funções são:

- Exibir a interface da aplicação no navegador.
- Enviar requisições HTTP para o contêiner do back-end.
- Receber e exibir as respostas vindas da API.

O front-end roda em uma porta diferente (geralmente 80 ou 8080) e pode ser acessado diretamente pelo navegador do usuário.

Comunicação entre os contêineres

Os dois contêineres se comunicam por meio de uma **rede interna criada pelo Docker**.

Dentro dessa rede, cada contêiner é identificado por um nome (por exemplo, "api" e "front"), permitindo que o front-end envie requisições diretamente ao back-end, sem depender de endereços externos.

Essa comunicação interna é isolada do ambiente externo, garantindo segurança e portabilidade.

Orquestração com Docker Compose

O **Docker Compose** é uma ferramenta que permite definir e executar vários contêineres de uma aplicação ao mesmo tempo.

Com ele, é possível descrever toda a estrutura da aplicação — como os serviços de front e back — em um único arquivo de configuração.

Ao iniciar o projeto com o Docker Compose, ele cria automaticamente:

- Um contêiner para o back-end.
- Um contêiner para o front-end.
- Uma rede interna que permite a comunicação entre eles.

Isso simplifica o processo de inicialização, pois ambos os serviços podem ser executados de forma integrada e coordenada.

Resumo

Componente	Função	Tipo
Back-End	Processa regras de negócio e responde às requisições	Contêiner 1
Front-End	Exibe a interface da aplicação e consome a API	Contêiner 2

Conclusão

No estado atual, o projeto possui **dois contêineres**:

um para o **back-end** e outro para o **front-end**.

Cada contêiner é responsável por um papel distinto dentro da aplicação e se comunica com o outro através de uma rede interna criada pelo Docker.

Se fosse adicionado um banco de dados, o projeto passaria a ter **três contêineres**, formando um ambiente completo de aplicação.

Imagens de Contêiner

O que é uma imagem de contêiner

Uma **imagem** é um **modelo pronto (ou receita)** usado para criar contêineres.

Ela contém:

- O sistema operacional mínimo necessário (geralmente Linux).
- As dependências que a aplicação precisa para funcionar.
- As instruções sobre como iniciar a aplicação.

Em outras palavras, a imagem é o **molde**, e o contêiner é a **instância em execução** desse molde.

Imagem do Contêiner do Back-End

A imagem usada para o back-end é baseada em **Node.js**, como por exemplo **node:18-alpine**.

Essa imagem já vem com:

- O interpretador do Node.js.
- O gerenciador de pacotes npm.
- Um ambiente Linux leve (a versão *alpine* é otimizada para ocupar pouco espaço).

O objetivo dessa imagem é **executar código JavaScript no servidor**.

Ela permite rodar um servidor Express, processar requisições, acessar bancos de dados e expor uma API REST.

Características principais:

- Voltada para **execução de aplicações Node.js**.
- Necessita das dependências do projeto instaladas (definidas no package.json).
- Executa um processo contínuo (por exemplo, npm start) que mantém o servidor ativo.

Em resumo, essa imagem é **dinâmica e voltada à lógica de negócio** da aplicação.

Imagem do Contêiner do Front-End

A imagem usada para o front-end é geralmente baseada em **Nginx**, como `nginx:alpine`.

Essa imagem contém:

- Um servidor web leve e estável (Nginx).
- Um sistema Linux reduzido.
- Configurações padrão para servir arquivos estáticos (HTML, CSS e JavaScript).

Ela não executa o código do front-end; apenas **distribui os arquivos** que já foram construídos (gerados pelo processo de build de um framework como React, Vue ou Angular).

Características principais:

- Voltada para **entrega de conteúdo estático**.
- É mais leve e simples, pois não processa lógica de aplicação.
- O foco é **performance e velocidade de entrega** dos arquivos no navegador.

Em resumo, essa imagem é **estática e voltada à interface com o usuário**.

Comparação Direta

Aspecto	Imagem do Back-End (Node.js)	Imagem do Front-End (Nginx)
Função principal	Executar código JavaScript no servidor	Servir arquivos estáticos da interface
Linguagem/tecnologia	Node.js / npm	Nginx (servidor web)
Tipo de aplicação	Dinâmica (processa requisições e respostas)	Estática (entrega conteúdo pronto)

Comportamento	Mantém um processo ativo (servidor)	Responde a requisições HTTP de arquivos
Peso	Médio (Node e dependências)	Leve (apenas servidor web)
Uso típico	APIs, microsserviços, back-end de aplicações	Front-end, páginas estáticas, SPAs construídas

Conclusão

Embora ambos sejam contêineres Docker, as **imagens utilizadas são diferentes porque têm finalidades distintas**:

- A **imagem do back-end** fornece o ambiente necessário para **executar código JavaScript no servidor**, como uma API Node.js.
- A **imagem do front-end** fornece o ambiente para **servir arquivos estáticos ao navegador**, sem processar lógica de negócio.

Em conjunto, essas duas imagens permitem que a aplicação funcione de ponta a ponta — o back-end cuidando do processamento e o front-end da interface com o usuário.

O papel do contêiner do banco de dados

O contêiner do **banco de dados** é responsável por armazenar e disponibilizar as informações utilizadas pela aplicação.

Enquanto o **back-end** executa regras de negócio e o **front-end** apresenta os dados ao usuário, o **banco de dados** mantém os registros de forma persistente.

Imagem do Contêiner do Banco de Dados

O banco de dados também é construído a partir de uma imagem oficial, como:

- mongo (para o MongoDB)
- mysql (para o MySQL)
- postgres (para o PostgreSQL)

Cada uma dessas imagens já vem com:

- O **sistema de gerenciamento de banco de dados** instalado.
- As **configurações padrão** para iniciar o serviço.
- Suporte a **armazenamento persistente** por meio de volumes, garantindo que os dados não se percam quando o contêiner é recriado.

Funções do contêiner do banco de dados

- **Gerenciar dados** estruturados ou não estruturados.
- **Receber conexões** do back-end para leitura e escrita de informações.
- **Armazenar** usuários, produtos, pedidos, sessões ou qualquer outro tipo de dado da aplicação.
- **Manter integridade e persistência** mesmo quando os outros contêineres são reiniciados.

O banco de dados **não se comunica diretamente com o front-end**; ele interage apenas com o **back-end**, que atua como intermediário.

Diferenças entre as imagens

Aspecto	Back-End (Node.js)	Front-End (Nginx)	Banco de Dados (Mongo/MySQL/Postgres)
Função principal	Processar regras de negócio e servir a API	Entregar arquivos estáticos (interface)	Armazenar e recuperar dados
Tipo de processamento	Lógico e dinâmico	Estático (servidor web)	Persistente (dados)
Comunicação	Recebe requisições do front-end	Consome a API do back-end	Recebe comandos do back-end

	e acessa o banco		
Estado	Efêmero (pode ser recriado facilmente)	Efêmero (sem necessidade de estado)	Necessita de persistência de dados
Imagem base	node:alpine ou similar	nginx:alpine	mongo, mysql, postgres etc.
Uso de volumes	Opcional	Não necessário	Fundamental para persistir dados

Relação entre os três contêineres

1. **Front-end** apresenta a interface ao usuário e envia requisições HTTP.
2. **Back-end** recebe as requisições, processa as regras de negócio e consulta o banco de dados.
3. **Banco de dados** armazena as informações e devolve os resultados ao back-end.

Esses três contêineres trabalham de forma **integrada**, cada um com sua responsabilidade isolada, mas comunicando-se por meio da **rede interna do Docker**.